# STAGE 5: Navigation & Routing

Flutter & Dart: From Zero to Expert — Stage Textbook 5 of 8

**Goal:** Confidently move between screens, pass data both ways, and structure multi-screen apps.
**Estimated time:** 2–3 weeks
**Prerequisite:** Stage 4 (Flutter State Management)

---

## 5.1 Basic Navigation: push & pop

```
// Going to a new screen
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const DetailScreen()),
);

// Going back
Navigator.pop(context);
```

**Key rules:**

- `Navigator.push()` adds a new screen on top of a stack; `Navigator.pop()` removes the current one and returns to the previous screen.
- Every Flutter app already manages a stack of screens for you — you're just pushing and popping from it.

---

## 5.2 Passing Data Forward and Back

```
// Forward: pass data via constructor
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => DetailScreen(itemId: 42)),
);

class DetailScreen extends StatelessWidget {
  final int itemId;
  const DetailScreen({super.key, required this.itemId});
  @override
  Widget build(BuildContext context) => Scaffold(body: Text('Item #$itemId'));
}

// Backward: return a result from the popped screen
Future<void> openForm(BuildContext context) async {
  final result = await Navigator.push<String>(
    context,
    MaterialPageRoute(builder: (context) => const FormScreen()),
  );
  if (result != null) {
    print('Form returned: $result');
  }
}

// Inside FormScreen:
Navigator.pop(context, 'Saved successfully'); // pops AND returns a value
```

---

## 5.3 Named Routes

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const HomeScreen(),
    '/details': (context) => const DetailScreen(),
    '/settings': (context) => const SettingsScreen(),
  },
);

// Navigating by name
Navigator.pushNamed(context, '/details');

// Passing arguments with named routes
Navigator.pushNamed(context, '/details', arguments: {'id': 42});

// Reading them on the receiving screen
final args = ModalRoute.of(context)!.settings.arguments as Map;
```

**Why it matters:** Named routes centralize all your app's screens in one place, which becomes essential once an app grows past 4–5 screens.

---

## 5.4 Modern Routing: go_router

Add to `pubspec.yaml`: `go_router: ^13.0.0`

```
import 'package:go_router/go_router.dart';

final router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (context, state) => const HomeScreen()),
    GoRoute(
      path: '/details/:id',
      builder: (context, state) {
        final id = state.pathParameters['id']!;
        return DetailScreen(itemId: int.parse(id));
      },
    ),
  ],
);

// In MaterialApp.router(routerConfig: router, ...)

// Navigating
context.go('/details/42');
```

**Why use go_router over the built-in Navigator:** it supports URL-based deep linking (critical for Flutter Web), nested navigation, and route guards (e.g., redirecting to a login screen) — all with a much smaller amount of boilerplate than Navigator 2.0's raw API.

---

## 5.5 Bottom Navigation & Tabs

```dart
class HomeShell extends StatefulWidget {
  const HomeShell({super.key});
  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  int _index = 0;
  final _screens = const [HomeTab(), SearchTab(), ProfileTab()];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens[_index],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _index,
        onTap: (i) => setState(() => _index = i),
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.search), label: 'Search'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profile'),
        ],
      ),
    );
  }
}
```

**Key rule:** Bottom navigation typically swaps the *body* of a single `Scaffold` rather than pushing new screens onto the navigation stack — it's a tab switch, not a navigation event.

---

## 5.6 Hands-On Project: Mini E-Commerce Navigation Flow

Build a Flutter app with the following screen flow:

1. A `HomeShell` with `BottomNavigationBar` for "Shop", "Cart", and "Account" tabs.
2. The "Shop" tab shows a product grid; tapping a product pushes a `ProductDetailScreen` (using `Navigator.push`, passing the product as a constructor argument).
3. `ProductDetailScreen` has a "Buy" button that pops back to Shop and returns a confirmation string, which is shown briefly as a `SnackBar`.
4. Convert at least the Shop → ProductDetail route to use **go_router** with a path like `/product/:id`.

This project combines push/pop, returning data, named or path-based routing, and tab-based navigation — all of Stage 5.

---

## 5.7 Stage 5 Test

**Question 1:** What is the difference between `Navigator.push()` and `Navigator.pushReplacement()`?

**Question 2:** How do you return a value from a screen when it is popped, and how do you receive that value on the screen that pushed it?

**Question 3:** What advantage does `go_router` have over the built-in `Navigator` for a Flutter Web app?

**Question 4:** In a bottom-navigation layout, why do tab switches typically avoid `Navigator.push()`?

**Question 5:** Write the `GoRoute` definition for a path `/user/:userId` that builds a `UserScreen` widget using the path parameter.

**Passing score:** Get all 5 correct before moving to Stage 6.